HW4 readme

Vinisha Bala Dhayanidhi

APPROACH

I did my best to try to simulate an arc-standard parser over each gold tree and let an oracle pick the transition at every step. A configuration is (stack, buffer, arcs) over word indices, with ROOT = 0 at the bottom of the stack (it has no input row, so it only appears as a head). I kept two jobs separate:

- apply_transition(config, t) updates a configuration and knows nothing about the gold tree. It raises an error on an illegal move (SHIFT on an empty buffer, an arc with under two stack items, or a LEFTARC that would give ROOT a head), so the oracle bugs fail loudly.
- Oracle.next_transition(config) reads the gold tree and only decides.

The oracle takes the first rule that applies with S1 the top of the stack and S2 the item below it:
1. (LEFTARC,l) if gold has S1 -> S2 (S2 not ROOT)
2. (RIGHTARC,l) if gold has S2 -> S1 AND S1 has already received all of its gold dependents
3. SHIFT otherwise

The root word gets (RIGHTARC,ROOT) since each arc's label is the gold label of its dependent. The output arcs come from these transitions, then sorted by word index (arcs are found in a different order than word order).

The key condition is the second half of rule 2. RIGHTARC pops S1, so a word must collect all of its dependents first. In "Book the flight through Houston", book -> flight is correct at [ROOT, book, flight], but flight has not yet attached Houston, so the oracle shifts. I count each head's gold dependents once per sentence and increment a counter when an arc is added. RIGHTARC is allowed only when the counts match instead of rescanning the arcs. LEFTARC does not need this check since, in a projective tree, anything between S2 and S1 was already attached before S1 was shifted.

RESULTS

- conll.simple: 100 sentences, 0 problematic. The arcs rebuilt from the transitions are identical to the input (diff shows no differences). The real time was about 0.05 s.
- toy.dep/toy.seq: my sequence matches toy.seq but the only difference is five trailing blank lines at the end of toy.seq (diff -B is clean), which I treated as an artifact of how the sample was produced.
- hw4_test_oracle.py (passes on patas): 20,000 random projective trees all round-trip exactly with 2n transitions each with the textbook example (J&M Fig. 20.7) matches and a hand-built non-projective tree is handled clean with a 5,000-word sentence that takes about 0.04 s.

I rebuilt the tree from the transitions and compared it to the input was my main check and it needs no known-good sequence, so I could run it on thousands of generated trees and lecture examples.

LIMITATIONS

- Projectivity: The arc-standard cannot build crossing arcs. If a gold tree is non-projective, the oracle can reach a state with nothing to reduce and an empty buffer. Then it just stops that sentence and warns on stderr instead of crashing and conll.simple does not trigger any warnings.
- One sequence per tree: many transition sequences can derive the same tree (spurious ambiguity). Deferring RIGHTARC until the head is complete picks one canonical sequence, so a classifier trained on it never sees the alternatives.

WHAT I LEARNED

The oracle is driven by knowing when a word is “finished”. The three transitions are simple, but the check that a word has all its dependents is what stops the greedy derivation from losing arcs, and it only needs local information plus a counter, so a training example comes out at every step in linear time.